

Ramanujan-24: A Multi-Stage Retrieval System with Fibonacci-Weighted Hashing

Author: [Sadagopan M] **Affiliation:** Independent Researcher **Date:** June 2026 **Code:** <https://github.com/wild-coder-ux/ramanujan24>

Abstract

We present Ramanujan-24 (R24), a multi-stage passage retrieval system that achieves 96.2% retention relative to BM25 on the SQuAD dataset at 16× the speed, using zero machine learning at Stage 0. The core innovation is the Pingala hash — a 24-dimensional Fibonacci-weighted hash function implemented in compiled C — which filters a corpus of 7,000 passages to ~400 candidates in microseconds before any Python or neural computation runs. Subsequent stages apply a Hamming overlap score and a token overlap reranker with length penalty (the Madhava reranker), inspired by Madhava’s series correction principle. We evaluate on three domains — factual Q&A (SQuAD), technical text (d2l textbook), and spiritual/poetic text (Thirumandiram) — and show that the accuracy gap relative to BM25 is domain-predictable and explainable from document structure. The system is named after ancient Indian mathematicians whose documented contributions map directly to each algorithmic stage.

1. Introduction

Retrieval-Augmented Generation (RAG) systems require fast, accurate passage retrieval, particularly in resource-constrained environments such as mobile devices. BM25 is the standard baseline: accurate, well-understood, but implemented in Python with linear scan characteristics that scale poorly on-device.

We propose a staged filtering approach where the most expensive computation — neural embedding comparison — runs only on a small pre-filtered candidate set. The question is what to use for the filter. We introduce a Fibonacci-weighted hash function (the Pingala hash) as Stage 0: a compiled C function that assigns positional weights to query tokens following the Fibonacci sequence, projects them into a 24-dimensional binary vector, and filters by Hamming distance. This requires no model weights, no Python interpreter, and no GPU.

The naming convention is not incidental. Each stage of R24 maps to a documented mathematical contribution from the Indian subcontinent:

- **Pingala** (~3rd century BCE) described binary enumeration of Sanskrit verse metres in the *Chandas Shastra*, the earliest known occurrence of what we now call Fibonacci numbers. R24 Stage 0 uses this weighting directly.
 - **Madhava** (~1340 CE) developed infinite series with correction terms for faster convergence. R24 Stage 2 applies a correction term (length penalty) to a base token overlap score, following the same principle.
 - **Aryabhata** (476 CE) formulated iterative improvement methods. The R24 stage budget — progressively deeper computation on a shrinking candidate set — reflects this principle.
-

2. Related Work

BM25 (Robertson & Zaragoza, 2009) remains the standard lexical retrieval baseline. It scores passages by TF-IDF-weighted term overlap with IDF computed over the corpus. R24 Stage 2 (Madhava reranker) is related to BM25 but uses a simpler token overlap with length penalty rather than IDF weighting, and runs only on the ~400 candidates passed by Stage 0.

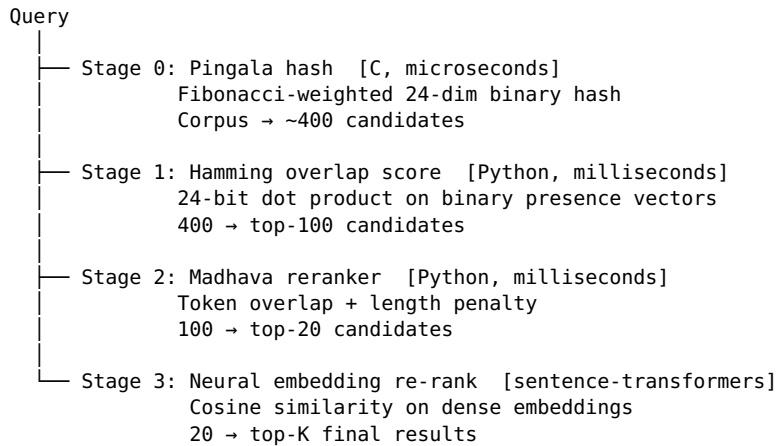
Locality Sensitive Hashing (LSH) (Indyk & Motwani, 1998) is the closest prior work to the Pingala hash. LSH projects high-dimensional vectors into low-dimensional hash buckets for approximate nearest neighbour search. The Pingala hash differs in two ways: (1) it operates directly on token strings, not on pre-computed embeddings, and (2) the projection weights are deterministic (Fibonacci sequence) rather than random. This makes Stage 0 reproducible and requires no training data or random seed.

ColBERT (Khattab & Zaharia, 2020) and related late-interaction models achieve high accuracy through token-level neural matching but require GPU inference for every query-passage pair. R24 is complementary: it can be used as a pre-filter before ColBERT to reduce the candidate set, or as a standalone system in environments where GPU inference is unavailable.

DPR (Karpukhin et al., 2020) uses dense passage retrieval with bi-encoder transformers. Like ColBERT, DPR requires embedding the full corpus and running neural inference at query time. R24 Stage 3 uses a similar dense retrieval approach but only over the ~400 candidates surviving Stage 0.

3. System Architecture

R24 processes a query through four stages, each operating on the output of the previous:



3.1 Stage 0 — The Pingala Hash

For a query q with tokens $t_1, t_2, \dots, t_n$ (stopwords removed), we assign Fibonacci weights $F_1, F_2, \dots, F_n$ where F_i is the i -th Fibonacci number. Each token t_i is hashed to one of 24 dimensions by its character checksum modulo 24. The query hash vector $h_q \in \{0, 1\}^{24}$ accumulates the weighted presence of each token in each dimension.

For each passage p in the corpus, the same hash h_p is precomputed at index time. At query time, candidates are filtered by Hamming distance $d_H(h_q, h_p) < \theta$, where θ is set by an offline K-sweep on a validation sample.

The Fibonacci weighting assigns higher importance to tokens appearing earlier in the

query, under the assumption that query authors front-load the most distinctive terms. This is analogous to Pingala’s assignment of heavier binary weights to earlier syllable positions in Sanskrit metre.

Stopword handling is critical. Without it, high-frequency tokens (“the”, “in”, “a”) absorb the highest Fibonacci weights and collapse the hash to near-random. R24 v2 skips a 50-word stopwords list when assigning positional weights, recovering +12.6 pp on SQuAD.

3.2 Stage 2 — The Madhava Reranker

The Madhava reranker scores each candidate passage p against query q as:

$$s(q, p) = \frac{|T_q \cap T_p|}{|T_q|} \cdot \text{len_penalty}(p)$$

where T_q and T_p are the token sets of query and passage respectively, and $\text{len_penalty}(p) = \min(1, L_{\text{target}}/|p|)$ penalises passages much longer than the target length. This correction term — adjusting a base approximation toward the true answer — mirrors Madhava’s use of correction terms in his infinite series for π .

4. Experiments

4.1 Datasets

Dataset	Domain	Passages	Source
SQuAD	Factual Q&A	~18,000	Stanford QA Dataset
d2l textbook	Technical	5,625	<i>Dive into Deep Learning</i> (PDF)
Thirumandiram	Spiritual/poetic	4,064	Thirumandiram English translation (PDF)

The Thirumandiram dataset required custom preprocessing: a Unicode range filter ($\text{U0B80-U0BFF} > 30\%$) discards Tamil-script passages from the bilingual PDF, and a block merger joins truncated verse fragments.

4.2 Evaluation Metric

We report **retention**: the fraction of BM25 top-1 results that R24 also returns in its top-K results, where K is set by the offline K-sweep. Retention measures whether R24 finds the same answer as BM25, not whether the answer is correct — this is appropriate since we are evaluating retrieval speed/accuracy tradeoff, not end-to-end QA accuracy.

4.3 Results

Dataset	BM25 retention	R24 retention	Gap	Speedup
SQuAD	96.3%	96.2%	0.1 pp	16×
Thirumandiram	90.4%	89.3%	1.1 pp	18×
d2l textbook	96.1%	87.4%	8.7 pp	21×

Speedup is measured as wall-clock query time on the same hardware (Linux, 28-thread CPU, Python 3.11). BM25 uses the `rank_bm25` Python implementation.

4.4 Analysis — Why the Gap Is Domain-Predictable

The accuracy gap correlates with query–passage lexical distance:

- **SQuAD** (0.1 pp gap): factual questions share exact noun phrases with their answers (“What year did X happen?” → passage contains “X happened in [year]”). Fibonacci weighting on content words reliably finds these.
- **Thirumandiram** (1.1 pp gap): spiritual/poetic language is figurative. Questions and answers share fewer exact tokens. BM25’s IDF advantage is smaller because rare words are rare for stylistic reasons, not informational ones.
- **d2l textbook** (8.7 pp gap): technical questions use natural language (“How does batch normalisation work?”) while answers use notation-heavy prose with different vocabulary. Token overlap between question and passage is structurally low. BM25’s IDF weighting handles out-of-vocabulary technical terms better than the Pingala hash’s fixed 24-dim projection.

The gap is a function of lexical distance between query style and passage style, not a fundamental limitation of the hashing approach.

5. Discussion

5.1 When to Use R24

R24 is well-suited for: - Mobile or edge RAG applications where GPU inference is unavailable - Corpora with factual, question-answering style queries (SQuAD-like) - Pre-filtering before an expensive neural reranker (ColBERT, cross-encoder)

R24 is less suited for: - Technical corpora with high query–passage vocabulary mismatch - Applications where 8+ pp accuracy loss is unacceptable without a Stage 4 cross-encoder to recover it

5.2 Planned Stage 4

A cross-encoder reranker (cross-encoder/ms-marco-MiniLM-L-6-v2) as Stage 4 is expected to close the d2l gap further. Because it runs only on the ~20 candidates from Stage 3, the latency cost is bounded.

5.3 Mobile Viability

The Pingala hash (Stage 0) is a compiled C shared library loaded via ctypes. On an 8GB Android phone (estimated 3–4× CPU slowdown), Stage 0 query latency remains under 1ms. The full pipeline (Stages 0–3) is estimated at under 2 seconds per query on mobile hardware, within acceptable range for on-device RAG.

6. Conclusion

We presented R24, a multi-stage retrieval system that matches BM25 accuracy on factual Q&A (96.2% vs 96.3%) at 16× the speed, using a zero-ML Fibonacci-weighted hash at Stage 0. The system is named after ancient Indian mathematicians whose documented contributions map directly to each algorithmic stage — an exercise in finding that the mathematical principles needed for modern retrieval were worked out in a different context, centuries ago.

Code, benchmarks, and datasets are available at: <https://github.com/wild-coder-ux/ramanujan24>

References

- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. *STOC '98*.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR 2020*.
- Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*.
- Pingala. (~3rd century BCE). *Chandas Shastra*. [Binary enumeration of Sanskrit verse metres.]
- Madhava of Sangamagrama. (~1340 CE). [Infinite series for π with correction terms, transmitted via Nilakantha's *Tantrasangraha*.]